

Guia completo: Autenticacao e autorizacao em APIs FastAPI com bo

Objetivo: entender e implementar, do zero, autenticacao com senha hasheada, JWT e autorizacao por roles em uma API FastAPI.

Publico: pessoa que ja conhece Python/FastAPI basico, mas nunca implementou login seguro.

Referencias oficiais consultadas:

- FastAPI Security JWT: <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>
- FastAPI Dependencies: <https://fastapi.tiangolo.com/tutorial/dependencies/>
- FastAPI Global Dependencies: <https://fastapi.tiangolo.com/tutorial/dependencies/global-dependencies/>
- python-jose: <https://python-jose.readthedocs.io/>
- bcrypt: <https://pypi.org/project/bcrypt/>

1. A visao geral

Autenticacao responde: quem e voce-

Autorizacao responde: o que voce pode fazer-

Neste guia vamos construir este fluxo:

1. Usuario se cadastra com nome, e-mail e senha.
2. A API salva a senha com hash bcrypt, nunca em texto puro.
3. Usuario faz login enviando e-mail e senha.
4. API valida a senha com `bcrypt.checkpw`.
5. API gera um access token JWT de curta duracao.
6. API gera um refresh token JWT de duracao maior.
7. Cliente envia o access token no header Authorization.
8. API decodifica o token, identifica o usuario e consulta o banco.
9. Rotas protegidas liberam ou bloqueiam o acesso.
10. Rotas administrativas verificam tambem a role do usuario.

Header usado em rotas protegidas:

Authorization: Bearer <access_token>

Uma API segura nao confia apenas no fato de alguem conhecer uma URL. Cada rota sensivel precisa declarar explicitamente sua regra de acesso.

2. Conceitos fundamentais

2.1 Senha em texto puro

Senha em texto puro e a senha como o usuario digitou. Exemplo:

123456
minhaSenha@2026

Nunca salve isso no banco. Se o banco vazar, todas as senhas vazam imediatamente.

2.2 Hash de senha

Hash e uma transformacao de uma entrada em uma saida aparentemente aleatoria. Para senhas, usamos algoritmos lentos de proposito, como bcrypt, Argon2id ou scrypt.

Exemplo conceitual:

senha123 -> \$2b\$12\$QpRk...muitos_caracteres...

O hash não é criptografado. No login, você pega a senha digitada e pergunta ao bcrypt: esta senha corresponde a este hash-

2.3 Salt

Salt é um valor aleatório usado no processo de hash. Ele impede que duas pessoas com a mesma senha tenham o mesmo hash. No bcrypt, o salt já fica embutido no hash final.

2.4 JWT

JWT significa JSON Web Token. Ele tem três partes:

header.payload.signature

O payload pode conter claims como:

sub: dono do token, geralmente o id do usuário

role: permissão, por exemplo admin ou funcionário

type: access ou refresh

exp: data de expiração

JWT é assinado, não criptografado. O cliente consegue ler o payload. Por isso, nunca coloque senha, documento, segredo ou dados muito sensíveis dentro dele.

2.5 Access token

Token usado para acessar rotas protegidas. Deve durar pouco, como 15, 30 ou 60 minutos.

2.6 Refresh token

Token usado para renovar access tokens sem pedir senha novamente. Dura mais, por exemplo 7 dias. Em sistemas reais, o ideal é armazenar refresh tokens no banco para permitir revogação.

2.7 Role

Role é o papel do usuário. Exemplos:

admin

funcionário

aluno

professor

A role permite autorização. Usuário autenticado pode entrar em uma rota comum. Usuário admin pode entrar em uma rota administrativa.

3. Estrutura recomendada

Use separação por responsabilidade:

```
app/  
  core/  
    config.py      -> lê variáveis de ambiente  
    security.py    -> hash, verify, create/decode JWT  
    auth.py        -> Depende para usuário atual e role  
    database.py    -> sessão do banco  
  models/  
    user_model.py  
  repositories/  
    user_repository.py  
  services/  
    user_service.py  
  schemas/
```

```
user_schema.py
auth_schema.py
routers/
  auth_routes.py
  user_routes.py
  course_routes.py
main.py
```

security.py nao deve depender de rotas.

auth.py pode depender de FastAPI, banco e repository.

routers devem ser finos: recebem request, chamam services e aplicam Depends.

services cuidam de regras de negocio.

repositories cuidam de queries.

4. Dependencias

No requirements.txt, voce precisa de algo como:

```
fastapi
uvicorn
SQLAlchemy
pydantic-settings
python-jose
bcrypt
email-validator
```

Instalacao manual:

```
pip install fastapi uvicorn sqlalchemy pydantic-settings python-jose bcrypt email-validator
```

bcrypt sera usado para senhas.

python-jose sera usado para JWT.

pydantic-settings sera usado para ler .env.

email-validator e necessario quando voce usa EmailStr no Pydantic.

5. Arquivo .env

O .env deve usar sintaxe de variavel de ambiente, nao sintaxe Python.

Correto:

```
SECRET_KEY=uma_chave_grande_e_aleatoria
ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7
ALGORITHM=HS256
```

Errado:

```
ACCESS_TOKEN_EXPIRE_MINUTES: int = 30
ALGORITHM = HS256
```

Para gerar SECRET_KEY:

```
python -c "import secrets; print(secrets.token_hex(64))"
```

Regra importante: nao suba .env com segredo real para repositorio publico.

6. app/core/config.py

Codigo recomendado:

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    secret_key: str
    access_token_expire_minutes: int
    refresh_token_expire_days: int
    algorithm: str = "HS256"

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

settings = Settings()
```

Explicacao:

BaseSettings procura as variaveis no ambiente e no .env.
secret_key recebe SECRET_KEY.
access_token_expire_minutes recebe ACCESS_TOKEN_EXPIRE_MINUTES.
refresh_token_expire_days recebe REFRESH_TOKEN_EXPIRE_DAYS.
algorithm recebe ALGORITHM.

Se os nomes nao baterem, sua aplicacao quebra ao subir.

7. app/core/security.py

Este arquivo contem as funcoes criptograficas e de JWT.

```
from datetime import datetime, timedelta, timezone
from typing import Optional

import bcrypt
from jose import JWTError, jwt

from app.core.config import settings

def hash_password(password: str) -> str:
    password_bytes = password.encode("utf-8")
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password_bytes, salt)
    return hashed.decode("utf-8")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(
        plain_password.encode("utf-8"),
        hashed_password.encode("utf-8"),
    )

def create_token(data: dict, expires_delta: timedelta) -> str:
    payload = data.copy()
    payload["exp"] = datetime.now(timezone.utc) + expires_delta
    return jwt.encode(payload, settings.secret_key, algorithm=settings.algorithm)

def create_access_token(user_id: int, role: str) -> str:
    return create_token(
        {"sub": str(user_id), "role": role, "type": "access"},
        timedelta(minutes=settings.access_token_expire_minutes),
    )
```

```
def create_refresh_token(user_id: int) -> str:
    return create_token(
        {"sub": str(user_id), "type": "refresh"},
        timedelta(days=settings.refresh_token_expire_days),
    )

def decode_token(token: str) -> Optional[dict]:
    try:
        return jwt.decode(token, settings.secret_key, algorithms=[settings.algorithm])
    except JWTError:
        return None
```

Pontos importantes:

- hash_password e usado antes de salvar senha.
- verify_password e usado no login.
- create_access_token inclui role porque autorizacao depende dela.
- create_refresh_token nao precisa incluir role.
- decode_token valida assinatura e expiracao.
- Se o token estiver adulterado ou expirado, decode_token retorna None.

8. UserModel com role

Exemplo:

```
from sqlalchemy import Boolean, Column, Integer, String
from app.core.database import Base

class UserModel(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, nullable=False)
    password = Column("senha", String, nullable=False)
    active = Column(Boolean, default=True)
    role = Column(String, nullable=False, default="funcionario", server_default="funcionario")
```

active permite desativar usuario.

role permite autorizacao.

server_default ajuda o banco.

default ajuda o SQLAlchemy.

Se voce adicionou role depois da tabela existir, crie/rode migration:

```
alembic revision --autogenerate -m "add role to user model"
```

```
alembic upgrade head
```

9. Schemas de usuario

```
from typing import Literal
from pydantic import BaseModel, EmailStr

class CreateUser(BaseModel):
    name: str
    email: EmailStr
    password: str

class UpdateUser(BaseModel):
```

```

name: str | None = None
email: EmailStr | None = None
password: str | None = None
active: bool | None = None
role: Literal["admin", "funcionario"] | None = None

```

```

class ResponseUser(BaseModel):
    id: int
    name: str
    email: EmailStr
    active: bool
    role: str

    class Config:
        from_attributes = True

```

Nunca inclua password no ResponseUser. Nem senha pura, nem hash.

10. Repository: buscar por e-mail

O login precisa encontrar usuario por e-mail.

```

from sqlalchemy import select
from sqlalchemy.orm import Session
from app.models.user_model import UserModel

class UserRepository:
    def findById_users(self, user_id: int, db: Session) -> UserModel | None:
        stmt = select(UserModel).where(UserModel.id == user_id)
        return db.scalar(stmt)

    def findByEmail_users(self, email: str, db: Session) -> UserModel | None:
        stmt = select(UserModel).where(UserModel.email == email)
        return db.scalar(stmt)

    def create_users(self, user: UserModel, db: Session) -> UserModel:
        db.add(user)
        db.commit()
        db.refresh(user)
        return user

```

A repository nao deve verificar senha. Ela so consulta e persiste dados.

11. Service: hash antes de salvar

```

from fastapi import HTTPException, status
from sqlalchemy.orm import Session

from app.core.security import hash_password
from app.models.user_model import UserModel
from app.repositories.user_repository import UserRepository
from app.schemas.user_schema import CreateUser, UpdateUser

class UserService:
    def __init__(self):
        self.repository = UserRepository()

    def create_user(self, user_data: CreateUser, db: Session) -> UserModel:
        existing = self.repository.findByEmail_users(user_data.email, db)

```

```

    if existing:
        raise HTTPException(status_code=400, detail="E-mail ja cadastrado")

    user_dict = user_data.model_dump()
    user_dict["password"] = hash_password(user_dict["password"])
    user = UserModel(**user_dict)
    return self.repository.create_users(user, db)

def update_user(self, user_id: int, user_data: UpdateUser, db: Session) -> UserModel:
    user = self.repository.findById_users(user_id, db)
    if not user:
        raise HTTPException(status_code=404, detail="Usuario nao encontrado")

    update_data = user_data.model_dump(exclude_unset=True)
    if "password" in update_data:
        update_data["password"] = hash_password(update_data["password"])

    for field, value in update_data.items():
        setattr(user, field, value)

    return self.repository.update_users(user, db)

```

Se voce ja tem usuarios antigos com senha em texto puro, o login com bcrypt nao vai funcionar para eles. Recrie os usuarios ou atualize as senhas passando pelo fluxo com hash.

12. Schemas de autenticacao

Crie app/schemas/auth_schema.py:

```

from pydantic import BaseModel, EmailStr

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

class RefreshTokenRequest(BaseModel):
    refresh_token: str

class TokenResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "bearer"

```

LoginRequest e entrada do login.

RefreshTokenRequest e entrada da renovacao.

TokenResponse e saida padrao.

13. app/core/auth.py

Este arquivo conecta JWT ao sistema de dependencias do FastAPI.

```

from collections.abc import Callable

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.database import get_db
from app.core.security import decode_token
from app.models.user_model import UserModel

```

```

from app.repositories.user_repository import UserRepository

bearer_scheme = HTTPBearer()

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> UserModel:
    payload = decode_token(credentials.credentials)

    if not payload or payload.get("type") != "access":
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token invalido ou expirado",
            headers={"WWW-Authenticate": "Bearer"},
        )

    user_id = payload.get("sub")
    if not user_id:
        raise HTTPException(status_code=401, detail="Token invalido")

    user = UserRepository().findById_users(int(user_id), db)
    if not user or not user.active:
        raise HTTPException(status_code=401, detail="Usuario inativo ou nao encontrado")

    return user

def require_role(*roles: str) -> Callable:
    def role_checker(current_user: UserModel = Depends(get_current_user)) -> UserModel:
        if current_user.role not in roles:
            raise HTTPException(status_code=403, detail="Sem permissao")
        return current_user

    return role_checker

```

Como pensar nisso:

get_current_user transforma token em usuario.
 require_role transforma usuario em permissao.
 Depends executa essas funcoes antes da rota.

Por que buscar usuario no banco se o token ja tem sub e role-

Porque o usuario pode ter sido desativado depois do token ser emitido. Tambem porque a role pode ter mudado. O token e uma credencial temporaria, mas o banco e a fonte atual de verdade.

14. Rotas de auth

Crie app/routers/auth_routes.py:

```

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.core.database import get_db
from app.core.security import create_access_token, create_refresh_token, decode_token,
verify_password
from app.models.user_model import UserModel
from app.repositories.user_repository import UserRepository
from app.schemas.auth_schema import LoginRequest, RefreshTokenRequest, TokenResponse

```



```

from app.schemas.user_schema import ResponseUser

router = APIRouter(prefix="/auth", tags=["Auth"])

@router.post("/login", response_model=TokenResponse)
def login(credentials: LoginRequest, db: Session = Depends(get_db)):
    user = UserRepository().findByEmail_users(credentials.email, db)

    if not user or not verify_password(credentials.password, user.password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="E-mail ou senha invalidos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    if not user.active:
        raise HTTPException(status_code=403, detail="Usuario inativo")

    return TokenResponse(
        access_token=create_access_token(user.id, user.role),
        refresh_token=create_refresh_token(user.id),
    )

@router.post("/refresh", response_model=TokenResponse)
def refresh_token(token_data: RefreshTokenRequest, db: Session = Depends(get_db)):
    payload = decode_token(token_data.refresh_token)

    if not payload or payload.get("type") != "refresh":
        raise HTTPException(status_code=401, detail="Refresh token invalido ou expirado")

    user = UserRepository().findById_users(int(payload["sub"]), db)
    if not user or not user.active:
        raise HTTPException(status_code=401, detail="Usuario inativo ou nao encontrado")

    return TokenResponse(
        access_token=create_access_token(user.id, user.role),
        refresh_token=create_refresh_token(user.id),
    )

@router.get("/me", response_model=ResponseUser)
def me(current_user: UserModel = Depends(get_current_user)):
    return current_user

/login autentica.
/refresh renova tokens.
/me mostra quem esta logado.

```

Nao responda "e-mail nao encontrado" no login. Use mensagem generica para nao facilitar descoberta de usuarios cadastrados.

15. Registrar no main.py

```

from fastapi import FastAPI
from app.routers import auth_routes, course_routes, user_routes

app = FastAPI(title="Cadastro de Cursos")

app.include_router(auth_routes.router)
app.include_router(course_routes.router)
app.include_router(user_routes.router)

@app.get("/")
def health():

```

```
return {"message": "Hello World"}
```

Depois disso, /auth/login deve aparecer no Swagger.

16. Protecao de rotas

Regra sugerida:

POST /users/	publico
GET /users/	admin
GET /users/{id}	admin
PUT /users/{id}	admin
DELETE /users/{id}	admin
GET /courses/	usuario logado
GET /courses/{id}	usuario logado
POST /courses/	admin
PUT /courses/{id}	admin
DELETE /courses/{id}	admin

Exemplo de rota apenas autenticada:

```
@router.get("/", response_model=list[CourseResponse])
def findAll_Courses(
    db: Session = Depends(get_db),
    current_user = Depends(get_current_user),
):
    service = CourseService()
    return service.get_all_courses(db)
```

Exemplo de rota apenas admin:

```
@router.post("/", response_model=CourseResponse, status_code=status.HTTP_201_CREATED)
def create_course(
    course: CourseCreate,
    db: Session = Depends(get_db),
    current_user = Depends(require_role("admin")),
):
    service = CourseService()
    return service.create_course(course, db)
```

Mesmo que current_user nao apareca no corpo da funcao, ele e necessario para forcar a dependencia a rodar.

17. Primeiro admin

Se todo usuario nasce funcionario, voce precisa criar o primeiro admin.

Opcoes:

1. Criar usuario comum e alterar role direto no SQLite em desenvolvimento.
2. Criar seed inicial.
3. Criar comando administrativo.
4. Permitir role no cadastro apenas em ambiente local.

SQL conceitual:

```
UPDATE users SET role = 'admin' WHERE email = 'lucas@example.com';
```

Em producao, nunca deixe qualquer pessoa se cadastrar como admin.

18. Testes no Swagger

Rode:

```
uvicorn app.main:app --reload
```

Abra:

<http://127.0.0.1:8000/docs>

Fluxo:

1. POST /users/ cria usuario.
2. Promova o usuario para admin se necessario.
3. POST /auth/login retorna access_token e refresh_token.
4. Clique em Authorize.
5. Cole: Bearer <access_token>
6. Teste GET /courses/.
7. Teste POST /courses/.
8. Teste sem token.
9. Teste com usuario funcionario em rota admin.
10. Teste /auth/refresh com refresh token.

Resultados esperados:

Sem token: 401 ou 403, dependendo do esquema.

Token invalido: 401.

Token valido sem role: 403.

Token valido com role correta: sucesso.

19. Testes com cURL

Criar usuario:

```
curl -X POST http://127.0.0.1:8000/users/ ^  
-H "Content-Type: application/json" ^  
-d '{"name":"Lucas","email":"lucas@example.com","password":"123456"}'
```

Login:

```
curl -X POST http://127.0.0.1:8000/auth/login ^  
-H "Content-Type: application/json" ^  
-d '{"email":"lucas@example.com","password":"123456"}'
```

Rota protegida:

```
curl http://127.0.0.1:8000/courses/ ^  
-H "Authorization: Bearer SEU_ACCESS_TOKEN"
```

Refresh:

```
curl -X POST http://127.0.0.1:8000/auth/refresh ^  
-H "Content-Type: application/json" ^  
-d '{"refresh_token":"SEU_REFRESH_TOKEN"}'
```

20. Erros comuns

Erro: ModuleNotFoundError: No module named 'config'

Causa: import errado.

Solucao: use from app.core.config import settings.

Erro: senha correta nao faz login

Causas provaveis:

- senha antiga foi salva em texto puro;
- voce esqueceu de hashear no cadastro;
- esta comparando hash com hash, ou texto com texto, de forma errada.

Erro: token sempre invalido

Causas provaveis:

- SECRET_KEY mudou;
- ALGORITHM mudou;
- token expirou;
- token foi copiado incompleto;
- voce esta usando refresh token em rota que exige access token.

Erro: admin recebe 403

Causa: role no banco nao e admin.

Solucao: confira a coluna role.

Erro: ResponseValidationError em usuario

Causa: ResponseUser pede campo que nao existe no model ou no banco.

Solucao: rode migration ou ajuste schema.

21. Checklist completo

- [] Corrigir .env.
- [] Criar Settings em config.py.
- [] Implementar hash_password.
- [] Implementar verify_password.
- [] Implementar create_access_token.
- [] Implementar create_refresh_token.
- [] Implementar decode_token.
- [] Garantir coluna role no UserModel.
- [] Rodar migration da role.
- [] Remover password dos schemas de resposta.
- [] Hashear senha no create_user.
- [] Hashear senha no update_user.
- [] Buscar usuario por e-mail no repository.
- [] Criar LoginRequest.
- [] Criar RefreshTokenRequest.
- [] Criar TokenResponse.
- [] Criar get_current_user.
- [] Criar require_role.
- [] Criar /auth/login.
- [] Criar /auth/refresh.
- [] Criar /auth/me.
- [] Registrar auth_routes no main.py.
- [] Proteger rotas de courses.
- [] Proteger rotas de users.
- [] Criar/promover primeiro admin.
- [] Testar sem token.
- [] Testar com token valido.
- [] Testar usuario sem permissao.
- [] Testar refresh.

22. Melhorias de producao

Para producao, esta base deve evoluir:

1. Salvar refresh tokens no banco.
2. Revogar refresh token no logout.
3. Rotacionar refresh token a cada renovacao.
4. Usar HTTPS sempre.
5. Implementar rate limit no login.
6. Criar politica de senha.
7. Registrar tentativas suspeitas.
8. Separar roles e permissions em tabelas se crescer.
9. Criar testes automatizados.
10. Nao versionar segredo real.
11. Considerar Argon2id em novos projetos.
12. Usar scopes OAuth2 se a autorizacao ficar granular.

23. Modelo mental final

Pense assim:

hash_password protege o banco.
verify_password protege o login.
create_access_token cria uma identidade temporaria.
decode_token valida essa identidade.
get_current_user transforma token em usuario real.
require_role transforma usuario real em permissao.
Depends conecta essas regras nas rotas.

Se voce entende esse encadeamento, entende o nucleo da autenticao e autorizacao em APIs FastAPI.